typesense

Semantic Search

Copy Markdown

Typesense supports the ability to do semantic search out-of-the-box, using built-in Machine Learning models or you can also use external ML models like OpenAI, PaLM API and Vertex AI API (starting from `v0.25`).

Use Case

Semantic search helps retrieve results that are conceptually related to a user's query.

For eg, let's say a user types in "ocean", but your dataset only contains the keyword "sea", semantic search can help pull up the "sea" result, since "ocean" is related to "sea".

In this article, we'll take a simple products dataset to show you how to implement semantic search in a few steps.

Step 1: Create a collection

This is very similar to creating a regular collection, except for the addition of an auto-embedding field highlighted below.

S-BERT E-5 GTE MPNET OpenAI PaLM VertexAI

```
curl -X POST \
  'http://localhost:8108/collections' \
  -H 'Content-Type: application/json' \
  -H "X-TYPESENSE-API-KEY: ${TYPESENSE_API_KEY}" \
  -d '{
        "name": "products",
        "fields": [
          {
            "name": "product_name",
            "type": "string"
          },
```

```
        "embed": {
          "from": [
            "product_name"
          ],
          "model_config": {
            "model_name": "ts/all-MiniLM-L12-v2"
          }
        }
      }
    ]
  }'
```

We're using the built-in ML model `ts/all-MiniLM-L12-v2` (aka S-BERT) in this example to automatically generate embeddings from the `product_name` field in the documents we'll be adding to this collection.

You can also use OpenAI, PaLM API or Vertex AI API to have Typesense automatically make API calls out to these services for generating embeddings that will power the semantic search.

You can also use any of the built-in ML models in our HuggingFace repo repo by specifying it as `ts/<model-name>`.

Step 2: Index your JSON data

You can send your JSON data as usual into Typesense:

```
curl "http://localhost:8108/collections/products/documents/import?action=create" \
       -H "X-TYPESENSE-API-KEY: ${TYPESENSE_API_KEY}" \
       -H "Content-Type: text/plain" \
       -X POST \
       -d '{"product_name": "Cell phone"}
           {"product_name": "Laptop"}
           {"product_name": "Desktop"}
           {"product_name": "Printer"}
           {"product_name": "Keyboard"}
           {"product_name": "Monitor"}
           {"product_name": "Mouse"}'
```

And that's it! It's semantic search time.

Step 3: Semantic Search

To do a semantic search, we just have to add the `embedding` field to the `query_by` search parameter:

```
curl 'http://localhost:8108/multi_search' \
    -H "X-TYPESENSE-API-KEY: ${TYPESENSE_API_KEY}" \
    -X POST \
    -d '{
      "searches": [
        {
          "q": "device to type things on",
          "query_by": "embedding",
          "collection": "products",
          "prefix": "false",
          "exclude_fields": "embedding",
          "per_page": 1
        }
      ]
    }'
```

This will return the following result.

Notice how we searched for `device to type things on` and even though those keywords don't exist in our JSON dataset, Typesense was able to do a semantic search and return the result as `Keyboard` since it is conceptually related:

json


```
{
  "results": [
    {
      "facet_counts": [],
      "found": 1,
      "hits": [
        {
          "document": {
            "id": "4",
            "product_name": "Keyboard"
```

```
          "vector_distance": 0.38377559185028076
        }
      ],
      "out_of": 7,
      "page": 1,
      "request_params": {
        "collection_name": "products",
        "per_page": 1,
        "q": "device to type things on"
      },
      "search_cutoff": false,
      "search_time_ms": 16
    }
  ]
}
```


Hybrid Search

In many cases, you might want to combine keyword searches along with semantic search. We call this Hybrid Search.

You can do this in Typesense, by adding a keyword field name to `query_by`, in addition to the auto-embedding field:

```
curl 'http://localhost:8108/multi_search' \
    -H "X-TYPESENSE-API-KEY: ${TYPESENSE_API_KEY}" \
    -X POST \
    -d '{
      "searches": [
        {
          "query_by": "product_name,embedding",
          "q": "desktop copier",
          "collection": "products",
          "prefix": "false",
          "exclude_fields": "embedding",
          "per_page": 2
        }
      ]
    }'
```

typesense

followed by `Printer` which is a semantic match.

```json
{
  "results": [
    {
      "facet_counts": [],
      "found": 7,
      "hits": [
        {
          "document": {
            "id": "2",
            "product_name": "Desktop"
          },
          "highlight": {
            "product_name": {
              "matched_tokens": [
                "Desktop"
              ],
              "snippet": "<mark>Desktop</mark>"
            }
          },
          "highlights": [
            {
              "field": "product_name",
              "matched_tokens": [
                "Desktop"
              ],
              "snippet": "<mark>Desktop</mark>"
            }
          ],
          "hybrid_search_info": {
            "rank_fusion_score": 0.8500000238418579
          },
          "text_match": 1060320051,
          "text_match_info": {
            "best_field_score": "517734",
            "best_field_weight": 102,
            "fields_matched": 3,
            "score": "1060320051",
            "tokens_matched": 0
          },
          "vector_distance": 0.510231614112854
```

```
            "id": "3",
            "product_name": "Printer"
          },
          "highlight": {},
          "highlights": [],
          "hybrid_search_info": {
            "rank_fusion_score": 0.30000001192092896
          },
          "text_match": 0,
          "text_match_info": {
            "best_field_score": "0",
            "best_field_weight": 0,
            "fields_matched": 0,
            "score": "0",
            "tokens_matched": 0
          },
          "vector_distance": 0.4459354281425476
        }
      ],
      "out_of": 7,
      "page": 1,
      "request_params": {
        "collection_name": "products",
        "per_page": 2,
        "q": "desktop copier"
      },
      "search_cutoff": false,
      "search_time_ms": 22
    }
  ]
}
```


Re-ranking Hybrid Matches

When doing hybrid search, by default Typesense returns both keyword matches and semantic matches in the results. For example:

```sh
curl 'http://localhost:8108/multi_search' \
    -H "X-TYPESENSE-API-KEY: ${TYPESENSE_API_KEY}" \
    -X POST \
    -d '{
```

```
            "q": "desktop copier",
            "collection": "products",
            "prefix": "false",
            "exclude_fields": "embedding",
            "per_page": 2
          }
        ]
      }'
```

A search for "desktop copier" might return results like this:

json


```
{
  "hits": [
    {
      "document": {
        "id": "2",
        "product_name": "Desktop"
      },
      "highlight": {
        "product_name": {
          "matched_tokens": ["Desktop"],
          "snippet": "<mark>Desktop</mark>"
        }
      },
      "hybrid_search_info": {
        "rank_fusion_score": 0.8500000238418579
      },
      "text_match": 1060320051,
      "text_match_info": {
        "best_field_score": "517734"
      },
      "vector_distance": 0.510231614112854
    },
    {
      "document": {
        "id": "3",
        "product_name": "Printer"
      },
      "hybrid_search_info": {
        "rank_fusion_score": 0.30000001192092896
      },
      "text_match": 0,
```

```
        "vector_distance": 0.4459354281425476
      }
    ]
  }
```


Notice how:

- The first result "Desktop" is a keyword match (high text_match score)
- The second result "Printer" is a semantic match (low vector_distance but zero text_match)

By default:

- Documents found through keyword search but not through vector search will only have a text match score
- Documents found through vector search but not through keyword search will only have a vector distance score

You can optionally compute both scores for all matches by setting `rerank_hybrid_matches: true`. When enabled:

- Documents found only through keyword search will also get a vector distance score
- Documents found only through vector search will also get a text match score

Example with re-ranking enabled:

sh

```sh
curl 'http://localhost:8108/multi_search' \
    -H "X-TYPESENSE-API-KEY: ${TYPESENSE_API_KEY}" \
    -X POST \
    -d '{
          "searches": [
            {
              "collection": "products",
              "query_by": "embedding,product_name",
              "q": "desktop copier",
              "rerank_hybrid_matches": true,
              "vector_query": "embedding:([], alpha: 0.8)",
              "exclude_fields": "embedding"
            }
          ]
        }'
```

Pagination

For effective pagination with semantic or hybrid search, use the `k` parameter in `vector_search` to ground the search to a specific number of closest items:

```
curl 'http://localhost:8108/multi_search' \
  -H "X-TYPESENSE-API-KEY: ${TYPESENSE_API_KEY}" \
  -X POST \
  -d '{
    "searches": [
      {
        "q": "device to type things on",
        "query_by": "embedding",
        "collection": "products",
        "prefix": "false",
        "vector_query": "embedding:([], k: 200)",
        "exclude_fields": "embedding",
        "per_page": 10,
        "page": 1
      }
    ]
  }'
```

- The `vector_query` parameter is set to `embedding:([], k: 200)`, limiting the vector search to the 200 nearest-neighbor items.
- `per_page` is set to `10`, returning the top 10 results sorted by relevance.
- You can then use the `page` parameter to paginate through the results.

Optionally, you can use the `distance_threshold` parameter in `vector_query` to fine-tune semantic search results:

```
curl 'http://localhost:8108/multi_search' \
  -H "X-TYPESENSE-API-KEY: ${TYPESENSE_API_KEY}" \
  -X POST \
  -d '{
    "searches": [
      {
        "q": "device to type things on",
        "query_by": "embedding",
```

```
        "exclude_fields": "embedding",
      }
    ]
  }'
```

For more details on the `vector_query` parameter and its options, please refer to the API documentation for Vector Search.

Live Demo

Here's a live demo that shows you how to implement Semantic Search and Hybrid Search.

You'll find the source code linked in the description.

Read the full API reference documentation about Vector Search here.

> **Note: CPU Usage**
>
> Built-in Machine Learning models are computationally intensive.
>
> So depending on the size of your dataset, when you enable semantic search and use a built-in ML model, even a few thousand records could take 10s of minutes to generate embeddings and index.
>
> If you want to speed this process up, you want to enable GPU Acceleration in Typesense.
>
> When you use a remote embedding service like OpenAI within Typesense, then you do not need a GPU, since the model runs on OpenAI's servers.

Edit page

Last Updated: 1/30/2025, 7:18:13 AM

← Search Analytics

Recommendations →